SQL PROJECT · #09

HR Analytics

27 KPIs business calculés en SQL

CE PROJET COUVRE

4 tables reliées en star schema

27 KPIs business prêts à copier

10 concepts SQL mobilisés

Compatible MySQL 8+ et PostgreSQL 13+

Portfolio-ready. Prêt à impressionner en entretien.

Laurent NZWENJA DJIAM

Data Analyst · Consultant Power BI

CONTEXTE DU PROJET

Le brief business

Une DRH d'entreprise moderne se pose 5 questions au quotidien : quel est notre taux d'attrition, où sont les foyers de sous-performance, sommes-nous équitables sur les salaires, sommes-nous en train de diversifier notre workforce, et où va notre budget bonus ?

Ce projet répond à ces questions par SQL en construisant un data model simple mais complet : employés, départements, présences, performance. Puis on écrit 27 requêtes qui alimentent un dashboard HR Analytics complet.

Le data model — Star schema

Quatre tables reliées, avec la table employees au centre :

- **employees** Table des salariés — pivot central du modèle.
- **departments** Table des unités organisationnelles — reliée par department_id.
- **attendance** Table des présences quotidiennes — reliée par employee_id.
- **performance** Table des évaluations annuelles — reliée par employee_id.

La stack technique

- SQL standard (MySQL 8+ / PostgreSQL 13+).
- Concepts couverts : JOIN, GROUP BY, HAVING, CASE WHEN, subqueries, CTE, window functions (RANK, DENSE_RANK, NTILE, OVER PARTITION BY), date functions.

SETUP — SCHÉMA & DONNÉES

Création des tables

KPI #00 • SETUP • Création de la base et des 4 tables

Concept SQL : DDL

```
CREATE DATABASE IF NOT EXISTS hr_analytics_db;
USE hr_analytics_db;

CREATE TABLE departments (
    department_id   INT PRIMARY KEY,
    department_name VARCHAR(100)
);

CREATE TABLE employees (
    employee_id       INT PRIMARY KEY,
    employee_name     VARCHAR(100),
    gender            VARCHAR(10),
    age               INT,
    department_id     INT,
    designation       VARCHAR(100),
    salary            DECIMAL(10,2),
    hire_date         DATE,
    city              VARCHAR(50),
    employment_status VARCHAR(20),
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

CREATE TABLE attendance (
    attendance_id   INT PRIMARY KEY,
    employee_id     INT,
    attendance_date DATE,
    status          VARCHAR(20),
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
);

CREATE TABLE performance (
    review_id          INT PRIMARY KEY,
    employee_id        INT,
    review_year        INT,
    performance_rating  DECIMAL(3,2),
    bonus              DECIMAL(10,2),
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
);
```

Le jeu de données complet (15 employés, 5 départements, présences et performance) est inclus dans le fichier .sql fourni. Retrouvez-le en scrollant plus bas dans ce PDF.

BLOC A · DÉMOGRAPHIE (5 KPIS)

KPI #01 · Nombre total d'employés

Concept SQL : COUNT

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

KPI #02 · Répartition Active vs Resigned

Concept SQL : GROUP BY + window function

```
SELECT
    employment_status,
    COUNT(*)                                AS nb_employees,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*))
          OVER (), 2)                       AS pct_total
FROM employees
GROUP BY employment_status;
```

KPI #03 · Ratio de diversité de genre

Concept SQL : GROUP BY + %

```
SELECT
    gender,
    COUNT(*)                                AS nb_employees,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*))
          OVER (), 2)                       AS pct_workforce
FROM employees
GROUP BY gender;
```

KPI #04 · Distribution par tranche d'âge

Concept SQL : CASE WHEN

```
SELECT
    CASE
        WHEN age < 30               THEN '< 30 ans'
        WHEN age BETWEEN 30 AND 39  THEN '30-39 ans'
        WHEN age BETWEEN 40 AND 49  THEN '40-49 ans'
        ELSE                            '50+ ans'
    END                                 AS age_group,
    COUNT(*)                                AS nb_employees
FROM employees
GROUP BY age_group
ORDER BY nb_employees DESC;
```

KPI #05 · Répartition par ville

Concept SQL : GROUP BY

```
SELECT
    city,
    COUNT(*)                                AS nb_employees
FROM employees
GROUP BY city
ORDER BY nb_employees DESC;
```

BLOC B • ANALYSE DES SALAIRES (5 KPIS)

KPI #06 • Salaire moyen global

Concept SQL : AVG + MIN + MAX + STDDEV

```
SELECT
    ROUND(AVG(salary), 2)   AS avg_salary_overall,
    MIN(salary)             AS min_salary,
    MAX(salary)             AS max_salary,
    ROUND(STDDEV(salary),2) AS salary_std
FROM employees;
```

KPI #07 • Salaire moyen par département

Concept SQL : JOIN + AVG

```
SELECT
    d.department_name,
    COUNT(e.employee_id)          AS nb_employees,
    ROUND(AVG(e.salary), 2)       AS avg_salary,
    ROUND(SUM(e.salary), 2)       AS total_payroll
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_name
ORDER BY avg_salary DESC;
```

KPI #08 • Salaire moyen par intitulé de poste

Concept SQL : GROUP BY sur designation

```
SELECT
    designation,
    COUNT(*)                    AS nb_employees,
    ROUND(AVG(salary), 2)       AS avg_salary
FROM employees
GROUP BY designation
ORDER BY avg_salary DESC;
```

KPI #09 • Département le mieux rémunéré

Concept SQL : Subquery + LIMIT

```
SELECT department_name, avg_salary
FROM (
    SELECT
        d.department_name,
        AVG(e.salary) AS avg_salary
    FROM employees e
    JOIN departments d ON e.department_id = d.department_id
    GROUP BY d.department_name
) t
ORDER BY avg_salary DESC
LIMIT 1;
```

KPI #10 • Quartiles de salaire

Concept SQL : Window function NTILE

```
SELECT
    employee_name,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile
FROM employees;
```

BLOC C · ATTRITION (3 KPIS)

KPI #11 · Taux d'attrition global

Concept SQL : CASE WHEN + AGG

```
SELECT
    ROUND(
        SUM(CASE WHEN employment_status = 'Resigned' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    )  AS attrition_rate_pct
FROM employees;
```

KPI #12 · Taux d'attrition par département

Concept SQL : JOIN + CASE WHEN

```
SELECT
    d.department_name,
    COUNT(*)                                            AS nb_employees,
    SUM(CASE WHEN e.employment_status = 'Resigned' THEN 1 ELSE 0 END) AS nb_resigned,
    ROUND(
        SUM(CASE WHEN e.employment_status = 'Resigned' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    )  AS attrition_rate_pct
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_name
ORDER BY attrition_rate_pct DESC;
```

KPI #13 · Attrition par année d'embauche

Concept SQL : EXTRACT + CASE WHEN

```
SELECT
    EXTRACT(YEAR FROM hire_date)                        AS hire_year,
    COUNT(*)                                            AS nb_hired,
    SUM(CASE WHEN employment_status = 'Resigned' THEN 1 ELSE 0 END) AS nb_resigned
FROM employees
GROUP BY hire_year
ORDER BY hire_year DESC;
```

BLOC D • PERFORMANCE (4 KPIS)

KPI #14 • Note moyenne de performance et bonus

Concept SQL : AVG

```
SELECT
    review_year,
    ROUND(AVG(performance_rating), 2) AS avg_rating,
    ROUND(AVG(bonus), 2)              AS avg_bonus
FROM performance
GROUP BY review_year;
```

KPI #15 • Top 5 performers

Concept SQL : CTE + RANK window function

```
WITH ranked_perf AS (
    SELECT
        e.employee_name,
        d.department_name,
        p.performance_rating,
        p.bonus,
        RANK() OVER (ORDER BY p.performance_rating DESC) AS rank_perf
    FROM performance p
    JOIN employees   e ON p.employee_id   = e.employee_id
    JOIN departments d ON e.department_id = d.department_id
)
SELECT *
FROM ranked_perf
WHERE rank_perf <= 5;
```

KPI #16 • Distribution du bonus par département

Concept SQL : JOIN + AVG + SUM

```
SELECT
    d.department_name,
    ROUND(AVG(p.bonus), 2)                AS avg_bonus,
    ROUND(SUM(p.bonus), 2)                AS total_bonus,
    MIN(p.bonus)                          AS min_bonus,
    MAX(p.bonus)                          AS max_bonus
FROM performance p
JOIN employees   e ON p.employee_id   = e.employee_id
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_name
ORDER BY total_bonus DESC;
```

KPI #17 • Répartition par tranche de rating

Concept SQL : CASE WHEN

```
SELECT
    CASE
        WHEN performance_rating >= 4.5 THEN 'Top performer'
        WHEN performance_rating >= 4.0 THEN 'High performer'
        WHEN performance_rating >= 3.5 THEN 'Solid performer'
        ELSE                            'À accompagner'
    END         AS performance_bucket,
```

```
    COUNT(*)     AS nb_employees
FROM performance
GROUP BY performance_bucket
ORDER BY nb_employees DESC;
```

BLOC E · EMBAUCHES ET ANCIENNETÉ (3 KPIS)

KPI #18 · Nouveaux embauchés par année

Concept SQL : EXTRACT YEAR

```
SELECT
    EXTRACT(YEAR FROM hire_date) AS hire_year,
    COUNT(*)                     AS new_hires
FROM employees
GROUP BY hire_year
ORDER BY hire_year;
```

KPI #19 · Ancienneté moyenne

Concept SQL : Date function DATEDIFF

```
-- MySQL
SELECT
    ROUND(AVG(
        DATEDIFF(CURDATE(), hire_date) / 365.25
    ), 1) AS avg_tenure_years
FROM employees
WHERE employment_status = 'Active';

-- PostgreSQL (variante)
-- SELECT ROUND(AVG(EXTRACT(YEAR FROM AGE(CURRENT_DATE, hire_date))), 1)
```

KPI #20 · Nouveaux embauchés par mois — trend

Concept SQL : EXTRACT YEAR + MONTH

```
SELECT
    EXTRACT(YEAR FROM hire_date)  AS annee,
    EXTRACT(MONTH FROM hire_date) AS mois,
    COUNT(*)                      AS new_hires
FROM employees
GROUP BY annee, mois
ORDER BY annee, mois;
```

BLOC F · PRÉSENCES ET ABSENTÉISME (3 KPIS)

KPI #21 · Taux de présence global

Concept SQL : CASE WHEN + AGG

```
SELECT
    ROUND(
        SUM(CASE WHEN status = 'Present' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    ) AS attendance_rate_pct
FROM attendance;
```

KPI #22 · Taux d'absentéisme par employé

Concept SQL : JOIN + CASE WHEN

```
SELECT
    e.employee_name,
    COUNT(a.attendance_id)                                  AS total_days,
    SUM(CASE WHEN a.status = 'Absent' THEN 1 ELSE 0 END)          AS days_absent,
    ROUND(
        SUM(CASE WHEN a.status = 'Absent' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(a.attendance_id), 2
    )                                                       AS absence_rate_pct
FROM employees e
JOIN attendance a ON e.employee_id = a.employee_id
GROUP BY e.employee_name
ORDER BY absence_rate_pct DESC;
```

KPI #23 · Utilisation des congés par département

Concept SQL : Multi-JOIN + CASE WHEN

```
SELECT
    d.department_name,
    COUNT(a.attendance_id)                                  AS total_days,
    SUM(CASE WHEN a.status = 'Leave' THEN 1 ELSE 0 END)          AS days_leave,
    ROUND(
        SUM(CASE WHEN a.status = 'Leave' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(a.attendance_id), 2
    )                                                       AS leave_rate_pct
FROM employees   e
JOIN attendance  a ON e.employee_id   = a.employee_id
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_name;
```

BLOC G • ANALYSES AVANCÉES (4 KPIS)

KPI #24 • Employés éligibles à une promotion

Concept SQL : Multi-JOIN + filtres avancés

```
SELECT
    e.employee_name,
    d.department_name,
    e.designation,
    p.performance_rating,
    ROUND(DATEDIFF(CURDATE(), e.hire_date) / 365.25, 1) AS tenure_years
FROM employees    e
JOIN performance p ON e.employee_id    = p.employee_id
JOIN departments d ON e.department_id = d.department_id
WHERE e.employment_status = 'Active'
  AND p.performance_rating >= 4.5
  AND DATEDIFF(CURDATE(), e.hire_date) / 365.25 >= 2
ORDER BY p.performance_rating DESC;
```

KPI #25 • Rang par salaire au sein du département

Concept SQL : Window function DENSE_RANK avec PARTITION BY

```
SELECT
    e.employee_name,
    d.department_name,
    e.salary,
    DENSE_RANK() OVER (
        PARTITION BY d.department_name
        ORDER BY e.salary DESC
    ) AS rank_in_dept
FROM employees    e
JOIN departments d ON e.department_id = d.department_id;
```

KPI #26 • Écart de salaire vs moyenne département

Concept SQL : Window function AVG OVER PARTITION

```
SELECT
    e.employee_name,
    d.department_name,
    e.salary,
    ROUND(AVG(e.salary) OVER (
        PARTITION BY d.department_name), 2)  AS avg_dept_salary,
    e.salary - AVG(e.salary) OVER (
        PARTITION BY d.department_name)      AS diff_vs_avg
FROM employees    e
JOIN departments d ON e.department_id = d.department_id
ORDER BY d.department_name, e.salary DESC;
```

KPI #27 • Dashboard HR — synthèse par département

Concept SQL : 2 CTEs + JOIN final

```
WITH dept_stats AS (
    SELECT
        d.department_id,
        d.department_name,
```

```
        COUNT(e.employee_id)                                    AS headcount,
        ROUND(AVG(e.salary), 2)                                 AS avg_salary,
        SUM(CASE WHEN e.employment_status = 'Resigned' THEN 1 ELSE 0 END) AS nb_resigned,
        ROUND(
            SUM(CASE WHEN e.employment_status = 'Resigned' THEN 1 ELSE 0 END)
            * 100.0 / COUNT(e.employee_id), 2
        )                                                       AS attrition_pct
    FROM departments d
    LEFT JOIN employees e ON e.department_id = d.department_id
    GROUP BY d.department_id, d.department_name
),
dept_perf AS (
    SELECT
        e.department_id,
        ROUND(AVG(p.performance_rating), 2) AS avg_rating,
        ROUND(SUM(p.bonus), 2)              AS total_bonus
    FROM performance p
    JOIN employees   e ON e.employee_id = p.employee_id
    GROUP BY e.department_id
)
SELECT
    s.department_name,
    s.headcount,
    s.avg_salary,
    s.attrition_pct,
    p.avg_rating,
    p.total_bonus
FROM dept_stats s
LEFT JOIN dept_perf p ON p.department_id = s.department_id
ORDER BY s.headcount DESC;
```

POUR ALLER PLUS LOIN

Faire tourner ce projet vous-même

Copiez-collez les blocs dans n'importe quel client SQL (DBeaver, MySQL Workbench, pgAdmin, DataGrip, ou même SQLite avec des ajustements mineurs). Le fichier .sql exécutable est également fourni séparément.

La suite de la série

Ce projet fait partie d'une série de 10 projets SQL portfolio :

- **Projet #10.** Real Estate Analytics — coming next.
- **Notification.** Suivez-moi sur LinkedIn pour être notifié dès la sortie.

Une idée d'évolution

Ce projet SQL peut être branché sur Power BI pour construire un dashboard interactif complet. J'ai un projet dédié à ce workflow (ContosoMarket) que je serais ravi de partager avec vous — envoyez-moi un mot.

Qui suis-je

Data Analyst / Consultant Power BI, 3+ ans d'expérience en environnement IT structuré (ATOS, DS TEAMS, Travel Planet). Stack : SQL, Power BI, DAX, Python, Azure (ADF, Databricks, ADLS). Certification Microsoft PL-300 en cours.

Actuellement en recherche active de missions Data Analyst / Consultant BI sur Paris — CDI ou freelance. Disponible ASAP.

CONTACT

Laurent NZWENJA DJIAM

Data Analyst · Consultant Power BI

laurentdjiam@gmail.com · +33 7 51 06 25 77

LinkedIn : Laurent NZWENJA DJIAM

Paris · Disponible ASAP

— Merci d'avoir lu. Bon code. —